thiscode Manual

Install + Core Features in 8 sections

Don't know a term? → docs/GLOSSARY.md (30+ terms explained)

Want a friendlier guide? → SETUP-BEGINNER.md (branching, gentle)

v1.0 unified first release

1. What is thiscode?

A bot-harness operations plugin that bundles Claude Code + Discord bots + Codex calls. Knowledge management and vault search are provided by the km plugin.

1.1. The one-line differentiator

ThisCode focuses on bot operations, installation, meetings, shared memory, and model routing; the km plugin owns knowledge management and vault-search fallback.

2. Install — ThisCode + the km plugin

```
# 1. Prerequisites (node 18+, jq, git)
# 2. ThisCode bot-harness install
git clone https://github.com/treylom/ThisCode ~/.claude/plugins/thiscode

# 3. Install the km plugin
claude plugin marketplace add treylom/tofukyung-plugins
claude plugin install km@tofukyung-plugins

# 4. In Claude Code, configure KM storage, MCP integrations, and settings: /km:setup
#    (This command does not install the search tiers.)

# 5. Tier 4 ripgrep (local tool)
bash ~/.claude/plugins/thiscode/scripts/install-ripgrep.sh --apply

# 6. Tier 2 Obsidian CLI detection and Obsidian app guidance (optional)
bash ~/.claude/plugins/thiscode/scripts/install-obsidian-cli.sh

# 7. Separate local embedding tool: vault-search MCP (optional; not a km tier)
bash ~/.claude/plugins/thiscode/scripts/install-vault-search.sh --apply

# 8. Tier 1 GraphRAG (optional; advanced)
bash ~/.claude/plugins/thiscode/scripts/install-graphrag.sh --apply

# 9. ThisCode bot-harness healthcheck
bash ~/.claude/plugins/thiscode/scripts/healthcheck.sh
```

ThisCode's scripts install the local search tools; the km plugin's `/km:search` runs the fallback. Run `/km:setup` separately in Claude Code to configure KM storage, MCP integrations, and settings.

For a branching, beginner-friendly guide: SETUP-BEGINNER.md

3. 4-Tier Search

ThisCode's `scripts/install-*.sh` install local tools; km's `/km:search` runs search; `/km:setup` configures storage, Obsidian MCP, and settings. The km order is GraphRAG → Obsidian CLI → Obsidian MCP → text search. Installing the separate vault-search MCP does not substitute for any km tier.

The figures below are old illustrative expectations, not measurements. They describe historical local tools, not the km search stages. The archived `benchmark/results/2026-05-13.json` skipped legacy engine IDs 1 (GraphRAG) and 2 (vault-search MCP); it does not validate these expectations or current km performance.

Historical local tool	Expected speed	Expected accuracy	Expected setup
GraphRAG (LLM + graph)	1500-3000ms	Very high	25 min
obsidian-cli (Obsidian index)	200-500ms	Medium	3 min
vault-search MCP (embedding)	500-1000ms	High	5 min
ripgrep (literal)	30-100ms	Low	0 min

For current search-stage conditions and execution, follow the km plugin's `/km:search`.

4. Knowledge Manager (km plugin)

Knowledge management and search are not bundled in ThisCode. Install the km plugin and use:

- `/km:search` — vault search and fallback
- `/km:knowledge-manager` — knowledge capture, classification, and storage
- `/km:setup` — create and update km plugin configuration

5. LLM model routing

After search, response generation auto-selects a model based on task complexity:

Task	Claude	Codex	Example
Simple	Haiku	gpt-5.5	“NuriFlow ARR”
Medium	Sonnet	gpt-5.5-codex	“Summarize the top 3 from the Q1 report”
Synthesis	Opus[1m]	gpt-5.5-codex-spark	“Infer the correlation between ARR and team size”

`scripts/route-model.mjs` uses a heuristic (query length + keywords). User override: `--model haiku|sonnet|opus`.

6. Meetings room / Codex Bridge / Shared memory / Hooks

- `/thiscode:meetings` — meeting-log folder + 4-file template, automatic
- `/codex` — bridge to OpenAI Codex (second opinion)
- shared-memory — 4-tier shared memory (T1 git / T2 machine / T3 project / T4 per-bot)
- SessionStart hook — auto-injects `soul.md`

7. 5-axis Benchmark

A 5-dimension measurement — latency_ms / recall_precision / cost_tokens / setup_time_min / kg_depth.

```
VAULT=~/your-vault bash benchmark/runners/run-all.sh
python3 benchmark/report-generator.py --print-only
```

For measurement methodology and how to interpret: BENCHMARK.md

8. FAQ + GLOSSARY references

Seven frequently asked questions: SETUP-BEGINNER.md FAQ section

Term glossary (LLM / MCP / CEL / embedding / precision / kg_depth / fallback / dispatcher / RAG, and more — 30+): GLOSSARY.md